Trabajo Práctico Obligatorio (TPO)

Materia: Aplicaciones Interactivas
Modalidad: Virtual
Año: 2026 – Primer cuatrimestre

Contenido

Trabajo Práctico Obligatorio (TPO) 1
1. Objetivo del trabajo práctico 2
2. Modalidad de trabajo 3
3. Tecnologías utilizadas 4
4. Módulos funcionales disponibles 5
5. Entregas 6
Entrega 1 – Backend funcional 7
Entrega 2 – Frontend React 8
Entrega 3 – Gestión de estado global y versión final 9
6. Rúbrica de evaluación 16

1. Objetivo del trabajo práctico

El Trabajo Práctico Obligatorio tiene como objetivo que los estudiantes desarrollen una **aplicación web completa**, integrando:

- desarrollo **backend con Spring Boot**
- **persistencia de datos con JPA**
- **seguridad con JWT**
- **frontend con React**
- **gestión de estado global con Redux**

Los estudiantes trabajarán sobre un **proyecto base provisto por la cátedra**, al cual deberán agregar nuevas funcionalidades.

Repo proyecto base:
https://github.com/Mancaceresuade/aplicacionesinteractivas_202601.git

El trabajo se desarrollará **de forma incremental durante el cuatrimestre**, en tres entregas.

---

2. Modalidad de trabajo

El trabajo se realizará en **grupos de 4 estudiantes**.

Cada grupo deberá seleccionar **uno de los módulos funcionales propuestos por la cátedra** y desarrollar las funcionalidades correspondientes.

Todos los grupos deberán:

- utilizar la **arquitectura base provista**
- mantener las **tecnologías definidas**
- respetar la **estructura del proyecto**

No está permitido cambiar el stack tecnológico.

---

3. Tecnologías utilizadas

Backend

- Java 21
- Spring Boot
- Maven
- JPA / Hibernate
- JWT

Frontend

- React
- React Router
- Redux Toolkit

Base de datos

- H2 o la provista en el proyecto base.

---

4. Módulos funcionales disponibles

Cada grupo deberá elegir **uno de los siguientes módulos**:

1 Gestión de mora
Permite registrar créditos en mora, asignar gestores y generar reportes de seguimiento.

2 Autorización de límite de crédito
Permite que operadores autoricen aumentos de límite de crédito en función del historial del cliente.

3 Marketing de renovación
Permite identificar créditos cancelados y generar campañas de renovación.

4 Sistema de comentarios
Permite registrar comentarios asociados a clientes, créditos o cobranzas.

5 Dashboard de estadísticas
Permite visualizar métricas del sistema mediante paneles de información.

6 Sistema de etiquetas para clientes
Permite clasificar clientes mediante etiquetas para análisis o segmentación.

Cada módulo deberá incluir:

- entidades
- endpoints REST
- interfaz de usuario
- integración frontend-backend

---

5. Entregas

El trabajo se desarrollará en **tres iteraciones**.

Entrega 1 – Backend funcional

📅 **Fecha:** 14 de abril

Contenidos evaluados:

- Arquitectura de backend
- Servicios REST
- Persistencia de datos
- Seguridad con JWT

Requisitos

El grupo deberá implementar en el backend:

- entidades JPA necesarias para el módulo elegido
- relaciones entre entidades
- repositorios
- servicios de negocio
- endpoints REST CRUD
- validaciones con Bean Validation
- autenticación con JWT
- manejo básico de errores

Entregables

Repositorio actualizado.

Archivo README que incluya:

- descripción del módulo
- modelo de datos
- endpoints implementados

Video corto (máx. 5 minutos) explicando:

- el modelo de datos
- los endpoints desarrollados
- funcionamiento general del backend.

Entrega 2 – Frontend React

📅 **Fecha:** 26 de mayo

Contenidos evaluados:

- desarrollo frontend
- componentes React
- conexión con backend

Requisitos

Desarrollar una interfaz en React que permita:

- visualizar información del módulo
- crear registros
- modificar registros
- eliminar registros

El frontend debe conectarse al backend mediante API REST.

Requisitos técnicos

- React
- React Router
- llamadas HTTP mediante fetch o axios
- manejo de errores básicos
- manejo de estados de carga

Entregables

Repositorio actualizado.

Video corto (máx. 5 minutos) mostrando:

- login en el sistema
- funcionamiento del frontend
- integración con el backend.

Entrega 3 – Gestión de estado global y versión final

📅 **Fecha:** 23 de junio

Objetivo

En esta entrega deben integrar gestión de estado global con **Redux Toolkit** y agregar un sistema de **roles y permisos** que afecte tanto al backend como al frontend.

El trabajo se divide en tres partes:

1. Migrar el frontend a Redux (si no lo hicieron en la Entrega 2)

2. Implementar el **Gestor de Permisos** (solo visible para ADMIN)

3. Implementar **Anular Crédito** y **Anular Cobranza** con sus reglas de negocio

Parte 1 – Redux Toolkit

Deben gestionar con Redux al menos:

- El **usuario autenticado** (`auth` slice): login, logout, registro

- La **información principal del módulo** que eligieron (al menos un slice con `createAsyncThunk`)

Cada operación asíncrona debe manejar los tres estados: `pending`, `fulfilled` y `rejected`.

Parte 2 – Gestor de Permisos

Descripción

El sistema tiene dos roles: `ADMIN` y `USER`. Los usuarios con rol `ADMIN` pueden acceder a una pantalla de gestión de permisos donde asignan capacidades específicas a los usuarios con rol `USER`.

Los permisos disponibles son:

| Permiso | Descripción |

|---------|----------|

| `puedeAnularCredito` | Permite anular un crédito |

| `puedeAnularCobranza` | Permite anular una cobranza del día |

Cambios en el backend

1. Modificar la entidad `Usuario`

Agregar dos campos booleanos a la entidad existente:

```java

private boolean puedeAnularCredito = false;

private boolean puedeAnularCobranza = false;

```

```

#### 2. Nuevo endpoint — Listar usuarios (solo ADMIN)
```
GET /api/admin/usuarios
```
- Requiere rol `ADMIN`
- Devuelve la lista de todos los usuarios con sus permisos actuales
- **No debe devolver la contraseña**

Respuesta esperada:
```json
[
  {
    "id": 1,
    "username": "juan",
    "rol": "USER",
    "puedeAnularCredito": false,
    "puedeAnularCobranza": true
  }
]
```
3. Nuevo endpoint — Actualizar permisos de un usuario (solo ADMIN)
```
PUT /api/admin/usuarios/{id}/permisos
```
- Requiere rol `ADMIN`
- Body:
```json
{
  "puedeAnularCredito": true,
  "puedeAnularCobranza": false
}
```
```

```

#### 4. Seguridad

Usar `@PreAuthorize` para proteger los endpoints del gestor:

```java

@PreAuthorize("hasRole('ADMIN')")

```

Para que `@PreAuthorize` funcione, deben habilitar `@EnableMethodSecurity` en `SecurityConfig`.

Frontend — Pantalla del Gestor de Permisos

- Ruta: `/admin/permisos`

- **Solo visible para usuarios con rol `ADMIN`**: el link en la Navbar no debe aparecer para usuarios `USER`, y la ruta debe estar protegida

- Muestra una tabla con todos los usuarios `USER` del sistema

- Cada fila tiene dos checkboxes: uno para `puedeAnularCredito` y otro para `puedeAnularCobranza`

- Al cambiar un checkbox, se llama al endpoint `PUT` inmediatamente

- El estado de esta pantalla debe estar en un **slice de Redux**

Parte 3 – Anular Crédito y Anular Cobranza

3.1 Anular Crédito

Regla de negocio

> **No se puede anular un crédito que tenga cobranzas registradas.**

> Si el crédito tiene al menos una cobranza asociada a cualquiera de sus cuotas, la anulación debe ser rechazada con un error claro.

Cambios en el backend

Nuevo campo en `Credito`:

```java

private boolean anulado = false;

```

Nuevo endpoint:

```

DELETE /api/creditos/{id}

```

- Requiere estar autenticado **y** tener el permiso `puedeAnularCredito`

- Valida que el crédito no tenga cobranzas → si tiene, lanza `BusinessException` con mensaje descriptivo

- Si la validación pasa, marca el crédito como `anulado = true` (no eliminar el registro de la base de datos)

Errores esperados:

```json

{

  "status": 400,

  "error": "Error de negocio",

  "mensajes": ["No se puede anular el crédito 3 porque tiene cobranzas registradas."],

  "timestamp": "2026-06-01T10:00:00"

}

```

```json

{

  "status": 403,

  "error": "Acceso denegado",

  "mensajes": ["No tiene permisos para anular créditos."],

  "timestamp": "2026-06-01T10:00:00"

}

```

Frontend

- En la página de **Créditos**, agregar un botón "Anular" por cada crédito

- El botón solo debe ser visible si `state.auth.user.puedeAnularCredito === true`

- Los créditos marcados como `anulado` deben mostrarse visualmente diferenciados (por ejemplo, tachados o con una etiqueta "ANULADO") y sin el botón de anular

- Al anular, mostrar el error del backend si la operación falla

3.2 Anular Cobranza

Regla de negocio

> **Solo se pueden anular cobranzas registradas el día de hoy.**
```

> Si la cobranza fue registrada en una fecha anterior, la anulación debe ser rechazada.

Cambios en el backend

Nuevo campo en `Cobranza`:

```java
private LocalDate fechaCobranza = LocalDate.now(); // asignar automáticamente al crear
private boolean anulada = false;
```

> Si `Cobranza` ya tiene un campo de fecha, reutilizarlo. Si no, agregarlo y asignarlo en el servicio al momento de crear la cobranza.

Nuevo endpoint:

```
DELETE /api/cobranzas/{id}
```

- Requiere estar autenticado **y** tener el permiso `puedeAnularCobranza`
- Valida que `fechaCobranza` sea igual a `LocalDate.now()` → si no, lanza `BusinessException`
- Si la validación pasa, marca la cobranza como `anulada = true` (no eliminar el registro)

Errores esperados:

```json
{
  "status": 400,
  "error": "Error de negocio",
  "mensajes": ["Solo se pueden anular cobranzas del día de hoy."],
  "timestamp": "2026-06-01T10:00:00"
}
```

```json
{
  "status": 403,
  "error": "Acceso denegado",
  "mensajes": ["No tiene permisos para anular cobranzas."],
  "timestamp": "2026-06-01T10:00:00"
```

}

```

#### Frontend

- En la página de **Cobranzas**, agregar un botón "Anular" por cada cobranza

- El botón solo debe ser visible si `state.auth.user.puedeAnularCobranza === true`

- Las cobranzas marcadas como `anulada` deben mostrarse diferenciadas y sin botón de anular

- Al anular, mostrar el error del backend si la operación falla

---

## Resumen de cambios esperados

### Backend

| Archivo | Cambio |

|---------|--------|

| `Usuario.java` | Agregar `puedeAnularCredito`, `puedeAnularCobranza` |

| `Credito.java` | Agregar campo `anulado` |

| `Cobranza.java` | Agregar campos `fechaCobranza` y `anulada` |

| `UsuarioRepository.java` | Método para buscar solo usuarios con rol `USER` |

| `UsuarioService` + `Impl` | Lógica de listar usuarios y actualizar permisos |

| `CreditoService` + `Impl` | Lógica de anulación con validación de cobranzas |

| `CobranzaService` + `Impl` | Lógica de anulación con validación de fecha |

| `AdminController.java` | Endpoints `GET` y `PUT` del gestor de permisos |

| `CreditoController.java` | Endpoint `DELETE /api/creditos/{id}` |

| `CobranzaController.java` | Endpoint `DELETE /api/cobranzas/{id}` |

| `SecurityConfig.java` | Habilitar `@EnableMethodSecurity` |

| DTOs nuevos | `UsuarioResponse`, `PermisosRequest` |

| `AuthResponse.java` | Agregar `puedeAnularCredito` y `puedeAnularCobranza` al token response |
```

Frontend

| Archivo | Cambio |

|---------|--------|

| `authSlice.js` | El `user` debe incluir `puedeAnularCredito` y `puedeAnularCobranza` |

| `permisosSlice.js` | Nuevo slice para el gestor de permisos |

| `Navbar.jsx` | Mostrar link a `/admin/permisos` solo si `user.rol === 'ADMIN'` |

| `PrivateRoute.jsx` | Extender para aceptar un `rol` requerido (o crear `AdminRoute`) |

| `Creditos.jsx` | Botón "Anular" condicional + estado visual de créditos anulados |

| `Cobranzas.jsx` | Botón "Anular" condicional + estado visual de cobranzas anuladas |

| `GestorPermisos.jsx` | Página nueva, solo para ADMIN |

| `api/admin.js` | Funciones fetch para los endpoints del gestor |

---

6. Rúbrica de evaluación

Criterio	Puntaje
Arquitectura backend	15
Modelado de entidades	10
Servicios y lógica de negocio	10
Seguridad con JWT	10
Frontend con React	15
Gestión de estado con Redux	10
Integración frontend-backend	10
Validaciones y manejo de errores	5
Calidad de código	5
Documentación	5
Presentación en video	5

Total: **100 puntos**

---

7. Consideraciones generales

- El trabajo es **obligatorio para aprobar la materia**.
- Todas las entregas deben realizarse dentro de las fechas establecidas.
- El código debe subirse al **repositorio del grupo**.
- Todos los integrantes deben participar en la presentación del video.